

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 10 实验名称 高效 IP 路由查找实验

一、实验内容

1. 实现最基本的前缀树查找算法。
2. 调研并实现某种优化的 IP 前缀查找方案。
 - 参考文献: Poptrie: A Compressed Trie with Population Count for Fast and Scalable Software IP Routing Table Lookup, SIGCOMM 2015。
3. 基于 forwarding-table.txt 数据集 (Network, Prefix Length, Port)
 - 以前缀树查找结果为基准, 检查所实现的 IP 前缀查找是否正确。
 - 对比基本前缀树和所实现 IP 前缀查找的性能, 如内存开销、平均单次查找时间等。

二、实验步骤

2.1 简单前缀树查找算法:

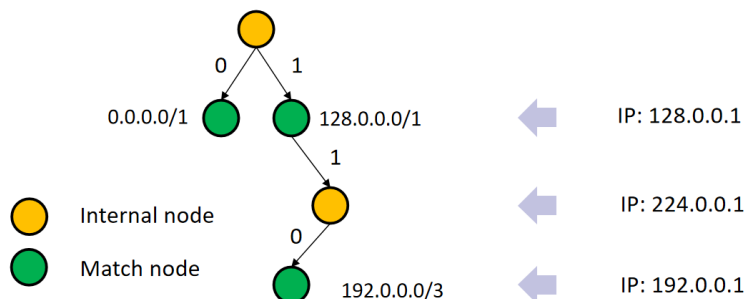


图 1: 简单前缀树结构示意图

简单前缀树查找的思路是利用二叉树的结构来存储 IP 的前缀信息, 将节点分为 Internal node 和 Match node 两种类型。Internal node 用于表示没有存储信息的中间节点, Match node 用于存储实际的前缀信息和对应的端口号。查找过程从根节点开始, 根据 IP 地址的每一位(0 或 1)沿着树的分支向下遍历, 在经过 Match node 时记录当前的匹配信息, 直到抵达叶子节点时结束查找, 返回最后记录的匹配信息作为结果。

因此, 我们需要构建这样一个数据结构来表示前缀树的节点:

```
typedef struct node{
    bool type; //I_NODE or M_NODE
    uint32_t port;
    struct node* lchild;
    struct node* rchild;
}node_t;
```

该数据结构包含节点类型、端口号以及左右子节点的指针。接下来,我们需要实现树的构建和查找功能。

首先是树的构造函数 `create_tree`:

```
void create_tree(const char* forward_file)
{
    FILE* fp = fopen(forward_file, "r");
    if (fp == NULL) {
        perror("Failed to open forwarding file");
        return;
    }

    uint32_t a, b, c, d, prefix_len, port;

    while (fscanf(fp, "%u.%u.%u.%u %u %u", &a, &b, &c, &d, &prefix_len, &port) == 6) {
        uint32_t ip = (a << 24) | (b << 16) | (c << 8) | d;

        node_t** current = &root_basic;

        for (int j = 0; j < prefix_len; j++) {
            int bit = (ip >> (31 - j)) & 1;

            if (*current == NULL) {
                *current = new_node();
            }

            current = (bit == 0) ? &((*current)->lchild) : &((*current)->rchild);
        }

        if (*current == NULL) {
            *current = new_node();
        }

        (*current)->type = M_NODE;
        (*current)->port = port;
    }

    fclose(fp);
}
```

由于在输入文件中,ip 地址,前缀长度和端口号是以 a.b.c.d e f 的形式存储的,因此我们按照这种格式读取数据,并将 ip 地址转换为 32 位整数。然后,根据前缀长度逐位遍历 ip 地址,构建前缀树。最后,在叶子节点处存储端口号。

随后是查找函数 `lookup_tree`:

```
uint32_t *lookup_tree(uint32_t* ip_vec)
{
    uint32_t* port_vec = (uint32_t*)malloc(sizeof(uint32_t) * TEST_SIZE);
    if (port_vec == NULL) {
        perror("Failed to allocate memory for port_vec");
        return NULL;
    }
}
```

```

}

for (int i = 0; i < TEST_SIZE; i++) {
    uint32_t ip = ip_vec[i];
    node_t* current = root_basic;
    uint32_t last_port = -1; // Default port if no match found

    for (int j = 0; j < 32; j++) {
        if (current == NULL) {
            break;
        }

        if (current->type == M_NODE) {
            last_port = current->port;
        }

        int bit = (ip >> (31 - j)) & 1;
        current = (bit == 0) ? current->lchild : current->rchild;
    }

    if (current != NULL && current->type == M_NODE) {
        last_port = current->port;
    }

    port_vec[i] = last_port;
}

return port_vec;
}

```

该函数接受一个 IP 地址数组作为输入，逐个查找每个 IP 地址在前缀树中的匹配端口号。查找过程中，记录最后一个匹配的端口号，并将结果存储在输出数组中返回。这样做的目的是为了可能存在的 Match node 接着 Match node 的情况，确保返回最长前缀匹配的结果。

2.2 Poptrie 方法优化前缀树查找：

Poptrie 方法来自于所给的论文，其主要思想是通过三个思想：

- 使用位图和紧凑数组来表示前缀树的节点，从而减少内存占用。
- 引入 Leaf Compression 压缩叶子节点，减少查找路径长度。
- 利用 Direct Pointing，使用大数组直接指向叶子节点，减少查找时间。

我们需要从这三个方面分别实现 Poptrie 方法。

首先，根据这个想法，我们需要设计一个数据结构来表示 Poptrie 的节点：

```

typedef struct {
    uint64_t vector;
    uint64_t leafvec;
    uint32_t base1;
}

```

```
uint32_t base0;
} __attribute__((packed)) poptrie_node_t;
```

该结构包含位图和基址信息,用于表示节点的子节点和叶子节点。这里我们说明一下使用位图的原因:

由于实际情况中,前缀树的节点通常是稀疏分布的,如果还是使用传统的指针方式存储子节点,会浪费大量内存。因此,我们只需要一个 64 位的位图用于表示子节点的存在情况,同时将子节点的信息存储在一个紧凑的数组中,当我们需要查询的时候,是需要使用位图指令 `popcnt` 来计算前面有多少个子节点,从而定位到紧凑数组中的位置。一方面这个指令的计算速度非常快,另一方面也大大节省了内存。

此处使用 `packed` 属性是为了强制紧凑排列,避免编译器对结构体进行对齐填充,从而减少内存占用。使用 64 位位图是因为寄存器通常是 64 位的,这样可以充分利用寄存器的位宽,提高查询效率,同时也可以尽可能减少树的高度。

同理对于 `leafvec`,是为了压缩叶子节点,避免连续的节点指向同一个端口号。此外,我们还需要一个大数组用于 `direct_array` 用来快速查找前 18 位前缀对应的节点。还要一个总的数据结构来存储 Poptrie 树的信息:

```
typedef struct{
    direct_ptr_t* direct_array;
    int s;

    poptrie_node_t* nodes;
    uint32_t node_count;
    uint32_t node_capacity;

    poptrie_leaf_t* leaves;
    uint32_t leaf_count;
    uint32_t leaf_capacity;
} poptrie_t;
```

接下来是 Poptrie 树的初始化,此处注意需要将 `capacity` 初始化为一个较大的值,不然后续还需要动态扩展。这里由于给出的数据集条目比较多,因此设置为 2000000。

对于 Poptrie 树的构建,比查找复杂很多。我们先调用前面的简单前缀树构建函数,得到一个完整的前缀树。然后把这个树转换为 Poptrie 树,这是一个递归的过程。

从根节点出发,对于 `depth` 小于 18 的节点,我们将其存储在 `direct_array` 中,并用最高位标记该节点是 Internal node 还是 Match node。对于 `depth` 大于等于 18 的节点,我们需要构建 Poptrie 节点,注意这里由于是 64 叉树,因此需要向下探 6 层。在构建 Poptrie 节点时,需要按 64 叉树向后预取二叉树的 6 层结构,据此生成 `vector` 位图来标记哪些分叉拥有子节点,同时生成 `leafvec` 位图来识别并压缩那些指向相同端口号的连续叶子节点。随后,利用 `popcnt` 指令计算这两个位图中 1 的数量,从而计算出子节点和叶子节点在全局数组中的基址索引(`base0` 和 `base1`),完成将稀疏的二叉树到紧凑 Poptrie 结构的映射。

对于查找则实现起来相对简单一点。首先按照高 18 位作为索引在 `direct_array` 中查找对应的节点,提取出最高位判断节点类型。如果是 Match node,则直接返回端口号;如果是 Internal node,则去掉最高位,得到 Poptrie 节点的索引。然后每次取 6 位作为子节点的索引,根据 `vector` 位图和 `base0` 计算出是否是子节点,若是则继续向下查找,否则根据 `leafvec` 和 `base1` 计算出端口号,返回。

代码比较长,这里就不全部贴出了,完整代码见文件。

三、实验结果和分析

3.1 平均查找时间

单次实验结果如下图所示：

```
kouyixin@ubuntu:~/CNLab/10-ip_lookup/code$ ./ip_trie_tree
Constructing the basic tree.....
Reading data from basic lookup table.....
Looking up the basic port.....
Constructing the advanced tree.....
Reading data from advanced lookup table.....
Looking up the advanced port.....
Dumping result.....
basic_pass-1
basic_lookup_time-22019us
advance_pass-1
advance_lookup_time-1753us
```

图 2: 实验结果

为保证结果的准确性，实验中我们多次运行查找函数，并取平均值作为最终结果。

```
kouyixin@ubuntu:~/CNLab/10-ip_lookup/code$ ./benchmark.sh
Run 38:
  basic_time: 17982us, advance_time: 1659us
Run 39:
  basic_time: 17359us, advance_time: 1721us
Run 40:
  basic_time: 17920us, advance_time: 1596us
Run 41:
  basic_time: 18032us, advance_time: 1735us
Run 42:
  basic_time: 18076us, advance_time: 1627us
Run 43:
  basic_time: 20521us, advance_time: 1646us
Run 44:
  basic_time: 16990us, advance_time: 1628us
Run 45:
  basic_time: 18533us, advance_time: 1567us
Run 46:
  basic_time: 18504us, advance_time: 1622us
Run 47:
  basic_time: 18313us, advance_time: 1801us
Run 48:
  basic_time: 18524us, advance_time: 2114us
Run 49:
  basic_time: 19036us, advance_time: 1588us
Run 50:
  basic_time: 18240us, advance_time: 1989us

Average basic_lookup_time: 18638us
Average advance_lookup_time: 2096us
```

图 3: 多次实验结果平均值

重复执行 50 次,得到均值如下:

- 基本前缀树查找平均时间:18638us
- Poptrie 前缀树查找平均时间:2096us

可以看到,Poptrie 方法相比于基本前缀树查找有了显著的提升,达到了近 9 倍的加速效果。

3.2 内存开销

3.2.1 basic

单个节点大小为 $8 + 8 + 8 = 24$ 字节(type + port + 2 个指针),测试集大约 70 万条路由,又由于大量的路由是有公共前缀的,估计节点数大约在路由条目的 2 倍,因此内存开销大约在 $24 * 140 \text{ 万} = 33.6\text{MB}$ 到 $24 * 210 \text{ 万} = 50.4\text{MB}$ 之间。

3.2.2 Poptrie

direct_array 大小为 $2^{18} \times 4 = 1\text{MB}$;

poptrie_node_t 大小为 $8 + 8 + 4 + 4 = 24$ 字节,参考论文数据,对于 50-70 万路由,Poptrie 节点数大约在 100,000 - 150,000 之间,因此内存开销大约在 $24 \times 15 \text{ 万} = 3.6\text{MB}$;

poptrie_leaf_t 大小为 4 字节,虽然原始路由有 70 万条,但经过 leafvec 压缩后,实际存储的独立叶子数会大幅减少。保守估算约 300,000 个叶子节点。此时开销大约为 $4 \times 30 \text{ 万} = 1.2\text{MB}$ 。故总的内存开销约为 $1\text{MB} + 3.6\text{MB} + 1.2\text{MB} = 5.8\text{MB}$ 。可以看出,Poptrie 方法在内存开销上也有显著优势,大约是基本前缀树的十分之一左右。

四、 实验中遇到的问题

在代码实现的过程中,主要遇到了两个小问题:

- 第一个是在初始化 Poptrie 树时,忘记将 capacity 初始化为一个较大的值,后续动态分配时也没有检查容量,导致数组越界,报错 segmentation fault。
- 第二个是在构建 Poptrie 节点时,默认输出端口号设置的是 0 而不是未查找到的标志-1,导致查找结果错误。通过输出报错位置的 ip 值和路由表进行比对,发现问题所在并修正。